



Instruction Sets Want to be Free!

Krste Asanovic

UC Berkeley, RISC-V Foundation, & SiFive Inc.

krste@berkeley.edu

www.riscv.org

Shanghai Jiaotong University, May 8, 2017





Why Instruction Set Architecture matters

- **Why can't Intel sell mobile chips?**
 - 99%+ of mobile phones/tablets based on ARM v7/v8 ISA
- **Why can't ARM partners sell servers?**
 - 99%+ of laptops/desktops/servers based on AMD64 ISA (over 95%+ built by Intel)
- **How can IBM still sell mainframes?**
 - IBM 360, oldest surviving ISA (50+ years)

Open Software/Standards Work!

<i>Field</i>	<i>Standard</i>	<i>Free, Open Impl.</i>	<i>Proprietary Impl.</i>
Networking	Ethernet, TCP/IP	Many	Many
OS	Posix	Linux, FreeBSD	M/S Windows
Compilers	C	gcc, LLVM	Intel icc, ARMcc
Databases	SQL	MySQL, PostgreSQL	Oracle 12C, M/S DB2
Graphics	OpenGL	Mesa3D	M/S DirectX
ISA	??????	-----	x86, ARM, IBM360

- Why not successful free & open standards and free & open implementations, like other fields
- Dominant proprietary ISAs are not great designs



What is RISC-V?

- Fifth generation of RISC design from UC Berkeley
- A high-quality, license-free, royalty-free RISC ISA specification
- Experiencing rapid uptake in both industry and academia
- Standard maintained by non-profit RISC-V Foundation
- Both proprietary and open-source core implementations
- Supported by growing shared software ecosystem
- Appropriate for all levels of computing system, from microcontrollers to supercomputers

Intel x86 “AAA” Instruction

- ASCII Adjust After Addition
- AL register is default source and destination
- If the low nibble is > 9 decimal, or the auxiliary carry flag $AF = 1$, then
 - Add 6 to low nibble of AL and discard overflow
 - Increment high byte of AL
 - Set CF and AF
- Else
 - $CF = AF = 0$
- Single byte instruction

ARM v7 LDMIAEQ Instruction

LDMIAEQ SP!, {R4-R7, PC}

- Load Multiple, Increment-Address
 - Writes to 7 registers from 6 loads
 - Only executes if EQ condition code is set
 - Writes to the PC (a conditional branch)
 - Can change instruction sets
-
- Idiom for "stack pop and return from a function call"



Why is name RISC-V? (pronounced “risk-five”)

RISC-I

RISC II

SOAR (aka RISC-III)

SPUR (aka RISC-IV)

RISC-V
(Raven-1,
28nm FDSOI,
2011)

Universal ISA Requirements

- Works well with existing software stacks, languages
- Is native hardware ISA, not virtual machine/ANDF
- Suits all sizes of processor, from smallest microcontroller to largest supercomputer
- Suits all implementation technologies, FPGA, ASIC, full-custom, future device technologies...
- Efficient for all microarchitecture styles: microcoded, in-order, decoupled, out-of-order, single-issue, superscalar, ...
- Supports extensive specialization to act as base for customized accelerators
- Stable: not changing, not disappearing



What's Different about RISC-V?

- *Simple*
 - Far smaller than other commercial ISAs
- *Clean-slate design*
 - Clear separation between user and privileged ISA
 - Avoids μ architecture or technology-dependent features
- A *modular* ISA
 - Small standard base ISA
 - Multiple standard extensions
- Designed for *extensibility/specialization*
 - Variable-length instruction encoding
 - Vast opcode space available for instruction-set extensions
- *Stable*
 - Base and standard extensions are frozen
 - Additions via optional extensions, not new versions

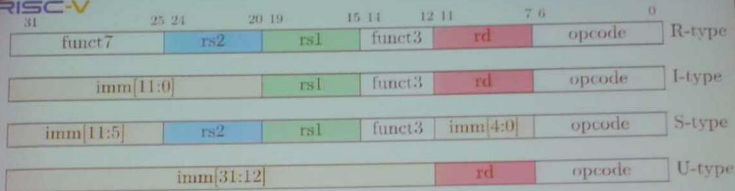


RISC-V Base Plus Standard Extensions

- Four base integer ISAs
 - RV32E, RV32I, RV64I, RV128I
 - RV32E is 16-register subset of RV32I
 - Only <50 hardware instructions needed for base
- Standard extensions
 - M: Integer multiply/divide
 - A: Atomic memory operations (AMOs + LR/SC)
 - F: Single-precision floating-point
 - D: Double-precision floating-point
 - G = IMAFD, "General-purpose" ISA
 - Q: Quad-precision floating-point
- All the above are a fairly standard RISC encoding in a fixed 32-bit instruction format
- Above user-level ISA components frozen in 2014
 - Supported forever after



RISC-V Standard Base ISA Details

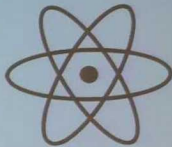


- 32-bit fixed-width, naturally aligned instructions
- 31 integer registers x1-x31, plus x0 zero register
- **rd/rs1/rs2** in fixed location, no implicit registers
- Immediate field (instr[31]) always sign-extended
- Floating-point adds f0-f31 registers plus FP CSR, also fused mul-add four-register format
- Designed to support PIC and dynamic linking

"A": Atomic Operations Extension

Two classes:

- Atomic Memory Operations (AMO)
 - Fetch-and-op,
op=ADD,OR,XOR,MAX,MIN,MAXU,MINU
- Load-Reserved/Store Conditional
 - With forward progress guarantee for short sequences
- All atomic operations can be annotated with two bits (Acquire/Release) to implement release consistency or sequential consistency
- *Current issues in memory model being resolved, will be stronger than pure relaxed model.*



Variable-Length Encoding

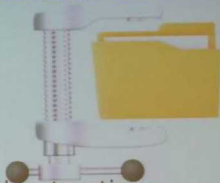
	xxxxxxxxxxxxxxxxxxaa	16-bit ($aa \neq 11$)
	xxxxxxxxxxxxxxxxxxxxxxx	32-bit ($bbb \neq 111$)
...xxxx	xxxxxxxxxxxxxxxxxxxxxxx	48-bit
...xxxx	xxxxxxxxxxxxxxxxxxxxxxx	64-bit
...xxxx	xxxxxxxxxxxxxxxxxxxxxxx	$(80 + 16 * nnn)$ -bit, $nnn \neq 111$
...xxxx	xxxxxxxxxxxxxxxxxxxxxxx	Reserved for ≥ 192 -bits

Byte Address: base+4 base+2 base

- Extensions can use any multiple of 16 bits as instruction length
- Branches/Jumps target 16-bit boundaries even in fixed 32-bit base
 - Consumes 1 extra bit of jump/branch address

"C": Compressed Instruction Extension

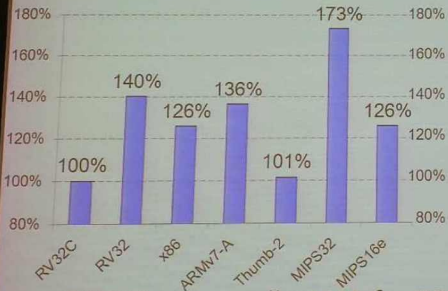
- Compressed code important for:
 - low-end embedded to save static code space
 - high-end commercial workloads to reduce cache footprint
- C extension adds 16-bit compressed instructions
 - 2-address forms with all 32 registers
 - 2/3-address forms with most frequent 8 registers
- 1 compressed instruction expands to 1 base instruction
 - Assembly lang. programmer & compiler oblivious
 - RVC $\Rightarrow$ RVI decoder only ~ 700 gates ($\sim 2\%$ of small core)
- All original 32-bit instructions retain encoding but now can be 16-bit aligned
- 50%-60% instructions compress $\Rightarrow$ 25%-30% smaller



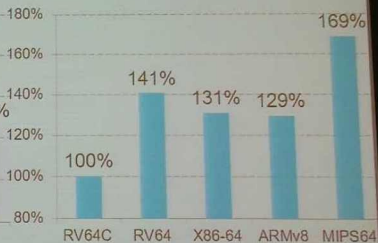


SPECint2006 compressed code size with save/restore optimization (relative to “standard” RVC)

32-bit Address



64-bit Address



- RISC-V now smallest ISA for 32- and 64-bit addresses
- All results with same GCC compiler and options



Proposed "V" Vector Extension State

Standard RISC-V scalar x and f registers

x31	f31
x1	f1
x0	f0

Vector
configuration

CSR

vcfg

Vector length

CSR

vlr

Up to 32 vector data registers, v0-v31, of at least 4 elements each, with variable bits/element (8,16,32,64,128)

v31[0]	v31[1]		v31[MVL-1]
v1[0]	v1[1]		v1[MVL-1]
v0[0]	v0[1]		v0[MVL-1]

MVL is maximum vector length, implementation and configuration dependent, but $MVL \geq 4$

p7[0]	p7[1]		p7[MVL-1]
p1[0]	p1[1]		p1[MVL-1]
p0[0]	p0[1]		p0[MVL-1]

8 vector predicate registers, with 1 bit per element



RISC-V Privileged Architecture

- Three privilege modes
 - User (U-mode)
 - Supervisor (S-mode)
 - Machine (M-mode)
- Supported combinations of modes:
 - M (simple embedded systems)
 - M, U (embedded systems with protection)
 - M, S, U (systems running Unix-style operating systems)
- Hypervisors to be run in modified S mode
 - Prioritize support for Type-2 Hypervisors like KVM
 - Can also support Type-1 Hypervisors in same model

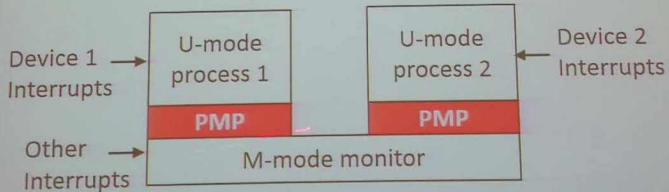


Simple Embedded Systems (M-mode only)

- No address translation/protection
 - “Mbare” bare-metal mode
 - Trap bad physical addresses precisely
- All code inherently trusted
- Low implementation cost
 - 2^7 bits of architectural state (in addition to user ISA)
 - $+2^7$ more bits for timers
 - $+2^7$ more for basic performance counters

Secure Embedded Systems (M, U modes)

- M-mode runs secure boot and runtime monitor
- Embedded code runs in U-mode
- Physical memory protection (PMP) on U-mode accesses
- Interrupt handling can be delegated to U-mode code
 - User-level interrupt support
- Provides arbitrary number of isolated subsystems



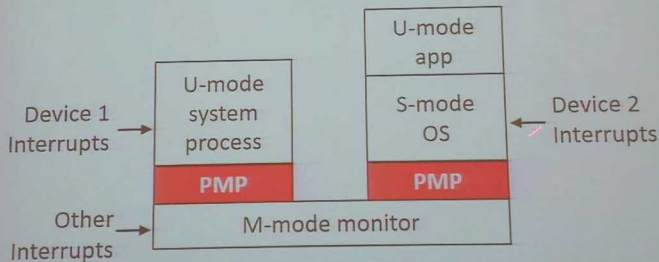


Virtual Memory Architectures (M, S, U modes)

- Designed to support current Unix-style operating systems
- Sv32 (RV32)
 - Demand-paged 32-bit virtual-address spaces
 - 2-level page table
 - 4 KiB pages, 4 MiB megapages
- Sv39 (RV64)
 - Demand-paged 39-bit virtual-address spaces
 - 3-level page table
 - 4 KiB pages, 2 MiB megapages, 1 GiB gigapages
- Sv48, Sv57, Sv64 (RV64)
 - Sv39 + 1/2/3 more page-table levels

S-Mode runs on top of M-mode

- M-mode runs secure boot and monitor
- S-mode runs OS
- U-mode runs application on top of OS or M-mode



RISC-V Virtualization Stacks

Application
ABI
AEE

Application	Application
ABI	ABI
OS	
SBI	
SEE	

Application	Application	Application	Application
ABI	ABI	ABI	ABI
OS		OS	
SBI		SBI	
Hypervisor			
HBI			
HEE			

- Provide clean split between layers of the software stack
- Application communicates with Application Execution Environment (AEE) via Application Binary Interface (ABI)
- OS communicates via Supervisor Execution Environment (SEE) via System Binary Interface (SBI)
- Hypervisor communicates via Hypervisor Binary Interface to Hypervisor Execution Environment
- All levels of ISA designed to support virtualization

Supervisor Binary Interface

- Platform-specific functionality abstracted behind SBI
 - Query physical memory map
 - Get device info
 - Get hardware thread ID and # of hardware threads
 - Save/restore coprocessor state
 - Query timer properties, set up timer interrupts
 - Send interprocessor interrupts
 - Send TLB shootdowns
 - Reboot/shutdown
- Simplifies hardware acceleration
- Simplifies virtualization



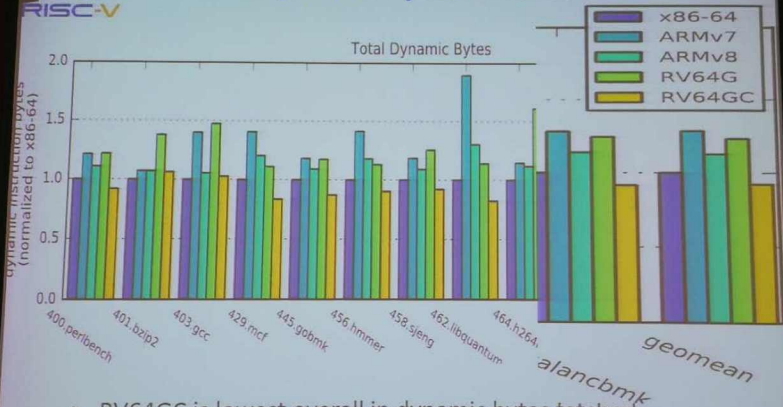
RV32I / RV64I / RV128I + M, A, F, D, Q, C

RISC-V "Green Card"

RISC-V Reference Card

Base Integer Instructions (RV32I/RV64I)				RV Unchanged Instructions (RV32I/RV64I)				Optional RV Extensions (RV32I/RV64I)				Optional Compound Instructions (RV32I/RV64I)			
Category	Name	Op	Op2	Category	Name	Op	Op2	Category	Name	Op	Op2	Category	Name	Op	Op2
Load	Load Byte	LB	rs1, rs2, imm	Shift	Shift Arithmetic	SRANDI	rs1, rs2, imm	Shift	Shift	SRANDI	rs1, rs2, imm	Store	Store Byte	SB	rs1, rs2, imm
	Load Halfword	LH	rs1, rs2, imm		Shift Right	SRANDI	rs1, rs2, imm		Shift	SRANDI	rs1, rs2, imm		Store Halfword	SH	rs1, rs2, imm
	Load Word	LW	rs1, rs2, imm		Shift Right Zero	SRANDI	rs1, rs2, imm		Shift	SRANDI	rs1, rs2, imm		Store Word	SW	rs1, rs2, imm
	Load Word Unaligned	LWU	rs1, rs2, imm		Shift Right Logical	SRANDI	rs1, rs2, imm		Shift	SRANDI	rs1, rs2, imm		Store Word Unaligned	SWU	rs1, rs2, imm
Store	Store Byte	SB	rs1, rs2, imm	Store	Store Halfword	SH	rs1, rs2, imm	Store	Store Word	SW	rs1, rs2, imm	Store	Store Word Unaligned	SWU	rs1, rs2, imm
	Store Halfword	SH	rs1, rs2, imm		Store Word	SW	rs1, rs2, imm		Store	SW	rs1, rs2, imm		Store	SWU	rs1, rs2, imm
	Store Word	SW	rs1, rs2, imm		Store Word Unaligned	SWU	rs1, rs2, imm		Store	SWU	rs1, rs2, imm		Store	SWU	rs1, rs2, imm
	Store Word Unaligned	SWU	rs1, rs2, imm		Store Word Unaligned	SWU	rs1, rs2, imm		Store	SWU	rs1, rs2, imm		Store	SWU	rs1, rs2, imm
Branch	Branch	BR	rs1, rs2, imm	Branch	Branch	BR	rs1, rs2, imm	Branch	Branch	BR	rs1, rs2, imm	Branch	Branch	BR	rs1, rs2, imm
	Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm
	Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm
	Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm		Branch	BR	rs1, rs2, imm
Op	Op	OP	rs1, rs2, imm	Op	Op	OP	rs1, rs2, imm	Op	Op	OP	rs1, rs2, imm	Op	Op	OP	rs1, rs2, imm
	Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm
	Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm
	Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm
Op	Op	OP	rs1, rs2, imm	Op	Op	OP	rs1, rs2, imm	Op	Op	OP	rs1, rs2, imm	Op	Op	OP	rs1, rs2, imm
	Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm
	Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm
	Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm		Op	OP	rs1, rs2, imm

Dynamic Bytes Fetched

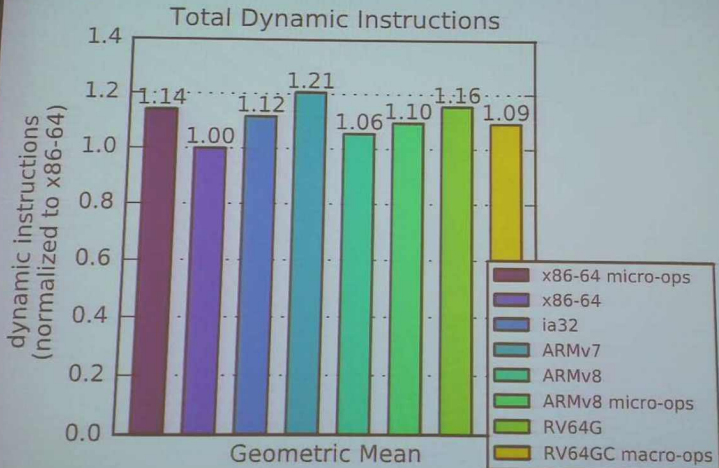


- RV64GC is lowest overall in dynamic bytes fetched
 - Despite current lack of support for vector operations

RISC-V Macro-Op Fusion Examples

- “Load effective address LEA” $\&(\text{array}[\text{offset}])$
`slli rd, rs1, {1,2,3}`
`add rd, rd, rs2`
- “indexed load” $M[\text{rs1}+\text{rs2}]$
`add rd, rs1, rs2`
`ld rd, 0(rd)`
- “clear upper word” $// \text{rd} = \text{rs1} \& 0\text{xffff_ffff}$
`slli rd, rs1, 32`
`srli rd, rd, 32`
- Can all be fused simply in decode stage
 - Many are expressible with 2-byte compressed instructions, so effectively just adds new 4-byte instructions
- RISC-V approach: prefer macroop fusion to larger ISA

RISC-V Competitive March Effort after Fusion

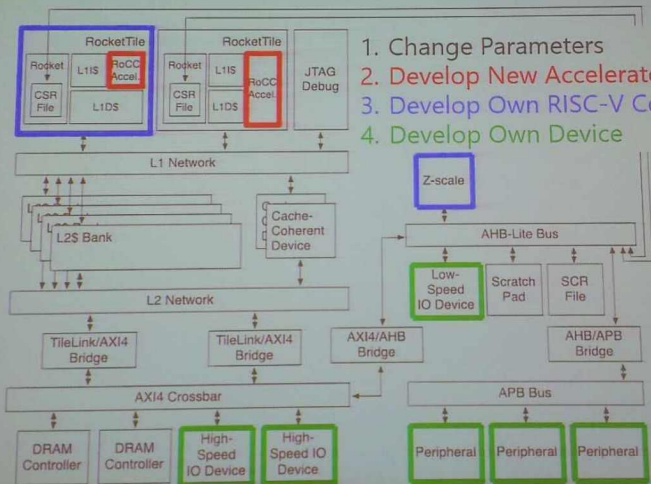


ARM Cortex-A5 vs. RISC-V Rocket

Category	ARM Cortex-A5	RISC-V Rocket
ISA	32-bit ARM v7	64-bit RISC-V v2
Architecture	Single-Issue In-Order	Single-Issue In-Order 5-stage
Performance	1.57 DMIPS/MHz	1.72 DMIPS/MHz
Process	TSMC 40GPLUS	TSMC 40GPLUS
Area w/o Caches	0.27 mm ²	0.14 mm ²
Area with 16K Caches	0.53 mm ²	0.39 mm ²
Area Efficiency	2.96 DMIPS/MHz/mm ²	4.41 DMIPS/MHz/mm ²
Frequency	>1GHz	>1GHz
Dynamic Power	<0.08 mW/MHz	0.034 mW/MHz

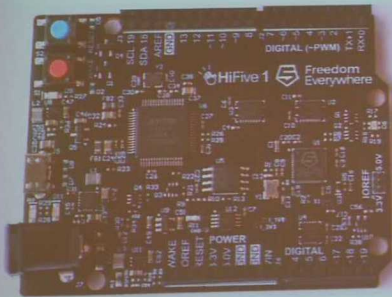
- PPA reporting conditions
 - 85% utilization, use Dhrystone for benchmark, frequency/power at TT 0.9V 25C, all regular VT transistors
- 10% higher in DMIPS/MHz, 49% more area-efficient

Rocket Chip Generator



1. Change Parameters
2. Develop New Accelerators
3. Develop Own RISC-V Core
4. Develop Own Device

siFive HiFive 1



- Open-Source RTL
- Arduino-Compatible
- Freedom E SDK
- Arduino IDE Environment
- Available for sale now!
- \$59

<https://www.crowdsupply.com/sifive/hifive>

1



RISC-V is GREAT at Perf and Power

Microcontroller	CPU Core	CPU ISA	CPU Speed	DMIPs/MHz	Total Dhrystones	DMIPs/mW
Intel Curie Module	Intel Quark SE	x86	32 MHz	1.3	41.6	0.35
ATmega328P	AVR	AVR (8-bit)	16 MHz	0.30	5	0.10
ATSAMD21G18	ARM Cortex M0+	ARMv6-M	48 MHz	0.93	44.64	
Nordic NRF51	ARM Cortex M0+	ARMv6-M	16 MHz	0.93	14.88	1.88
Freedom E310	SiFive E31	RISC-V RV32IMAC	200 MHz 320 MHz (max)	1.61	320.39	3.16





RISC-V Outside Berkeley

- Adopted as “standard ISA” for **India**
 - IIT-Madras \$90M funding to build 6 different open-source RISC-V cores, from microcontrollers to servers
 - C-DAC \$45M funding to build 2GHz quad-core
- **NVIDIA** selected RISC-V for on-chip microcontrollers
- **LowRISC** project based in Cambridge, UK producing open-source RISC-V Rocket-based SoCs
 - Led by Raspberry Pi co-founder, privately funded
- Many companies developing RISC-V cores for use in tightly integrated hardware/software IP blocks
- First commercial RISC-V cores have already shipped!
 - Rumble Development Corp, for dental camera/imaging
- Multiple commercial silicon implementations should be for sale later this year



Foundation Mission Statement

The RISC-V Foundation is a non-profit consortium chartered to standardize, protect, and promote the free and open RISC-V instruction set architecture together with its hardware and software ecosystem for use in all computing devices.



RISC-V Background

- In 2010, after many years and many projects using MIPS, SPARC, and x86 as basis of research, it was time for the Computer Science team at UC Berkeley to look at what ISAs to use for their next set of projects
- Obvious choices: x86 and ARM
 - x86 impossible – too complex, IP issues
 - ARM mostly impossible – complex, IP issues
- So UC Berkeley started “3-month project” during the summer of 2010 to develop their own clean-slate ISA



Foundation Principles

- The RISC-V ISA and related standards shall remain open and license-free to all parties. The standard specifications shall always be publicly available as an online download
- ^I The compatibility test suites shall always be publicly available as a source code download
- To protect the standard, only members (with commercial RISC-V products) of the Foundation in good standing can use “RISC-V” and associated trademarks, and only for devices that pass the tests in the open-source compatibility suites maintained by the Foundation



RISC-V is NOT an Open-Source Processor

- RISC-V is an ISA specification – NOT an open-source processor core
- ⁱ Most of the cost of chip design is in software, so we want to make sure software can be reused across many chip designs
- The Foundation will encourage both open-source and proprietary implementations of the RISC-V ISA specification



Foundation Function

- official source of information about RISC-V, maintains an online repository of RISC-V documents, and promotes adoption of RISC-V by organizing both online and live events
- responsible for sustaining, evolving and open-source licensing of the RISC-V instruction set architecture and surrounding hardware and software ecosystem over time in response to changes in technology and the needs and requests of the user community
- manages licensing of the RISC-V trademarks and provides a vehicle to decide whether a project or product can use the RISC-V trademark
- maintains a directory of public-domain instruction set architecture and micro-architectural techniques, culled from publications and expired patents
- produces and sells RISC-V promotional material, the proceeds from which are used to further the goals of the Foundation



Foundation Organization

- The Board of Directors consists of seven+ members, whose replacements are elected by the membership
- The Board is ultimately responsible for fulfilling the Mission Statement
- The Board can amend the By-Laws of the RISC-V foundation via a two-thirds affirmative vote
- The Board appoints chairs of ad-hoc committees to address issues concerning RISC-V, and has the final vote of approval of the recommendation of the ad-hoc committees.
- All members of committees must be members of the RISC-V Foundation
 - Committee chairs must report to the Board, and committees are subject to termination if the Board decides a committee is not making satisfactory progress
- Given the worldwide interest in RISC-V, the Board will appoint Chairs of Regional Committees to promote RISC-V development in local communities



Classes of Membership

- Platinum Sponsor dues of US\$25,000 per year
 - 10 complimentary registrations for RISC-V meetings during the year of membership and the most prominent and largest display of company logos in online and print materials for RISC-V
- Platinum members are eligible for Board seat elections and to Chair Foundation Technical Committees, Marketing Committees and Sub-Committees
- Gold Sponsor dues of US\$10,000 per year
 - 7 complimentary registrations for RISC-V meetings during the year of membership and prominent display of company logos in online and print materials for RISC-V
- Gold Sponsor members are eligible to Chair Foundation Technical Committees, Marketing Committees and Sub-Committees
- Silver Sponsor dues of US\$5,000 per year
 - 5 complimentary registrations for RISC-V meetings and for their logos to be displayed in RISC-V online and print materials
- All Sponsor organizations have one vote per open position in Board elections.
- Auditor dues of US\$2,500 per year (non-voting)
 - 2 complimentary registrations for RISC-V meetings during the year of membership and for their names to be listed in online and print materials for RISC-V
- Individual Member dues of US\$99 per year (non-voting)



Foundation Status

- Articles of Incorporation filed August, 2015
 - RISC-V Foundation Corporation is a legal, non-profit operating entity – 501(c)(6)
- 60 member companies have joined the Foundation
- Board of Directors formed in Q2, 2016
- Technical & Marketing Committees formed in Q3, 2016
 - Chairs & Vice-Chairs elected
- Membership Agreement & Bylaws ratified and released in Q4, 2016



Berkeley
Architecture
Research

Platinum Members

bluespec



cortus

DRAPER DOVER

MICROSYSTEMS

Google



HUAWEI



Mellanox
TECHNOLOGIES



Microsemi



Microsoft



NVIDIA

NXP

QUALCOMM

Rambus

SAMSUNG



SiFive



Western
Digital

Cryptography Research